吉林大学

软件学院

《面向对象程序课程设计》
实验报告

实验地点	计算机大楼实验室	**座位号**	
班级	**2 班**		
学号	**55250229**		
姓名	**李佳烨**		

2026—2027 学年第 1 学期

实验项目	**模拟即时通信系统实现**

一、设计任务分析

1 系统目标与功能需求

本项目使用 C++ 实现模拟即时通信系统，围绕同一用户在 QQ、微信和微博中的账号、好友与群关系进行管理。三个服务均为程序内的模拟服务，不连接真实社交平台。系统保留数字菜单的控制台入口，并提供 Qt 图形入口；课程核心是面向对象的数据管理，图形界面、SQLite 持久化和聊天功能作为扩展。

账号部分支持注册、登录、资料修改、服务开通与切换。QQ 与微博共用对外号码，微信使用独立号码并可与 QQ 建立绑定。好友部分提供查询、申请、同意、备注、删除，以及共同好友、跨服务推荐和同城推荐。群管理包含成员查询、申请与邀请、审批、退出、移除、管理员任免、群主转让、一级子群及群模式切换。

2 重点与技术难点

设计重点是将人员资料与服务身份分开，保证跨服务查询有明确的映射依据；把群数据与权限规则分开，使 QQ 和微信模式能够动态切换。实现难点包括成员关系一致性、失败操作不丢失输入、业务资料与聊天记录分开保存，以及网络消息分帧、后台收发和群历史补齐。

通信范围限定为 Windows 本机多进程。私聊采用真实 TCP 点对点连接，群聊通过独立本机服务转发并保存消息。当前未进行实体双机测试，不包含云端部署、音视频或与真实社交账号互通。系统目标是可运行、可解释、便于课程演示，避免超出任务范围的复杂基础设施。

二、设计方案

1 模块划分与对象关系

系统分为实体、业务、持久化、界面和通信五部分。PersonLJY 保存出生日期、所在地等人员资料；ServiceAccountLJY 保存服务类型、对外号码、昵称与申请日期，使用 personId 关联人员。PlatformLJY 集中持有人员、账号、关系和群容器。每个用户是类的对象，新增用户只增加数据，不新增一套源文件。

SessionManagerLJY 保存当前登录身份和服务确认状态，组合 AccountServiceLJY、FriendManagerLJY 与 GroupManagerLJY 执行业务。ConsoleAppLJY 和各 Qt 页面调用相同会话接口，分别处理输入与显示。业务核心不直接操作控件或拼接 SQL，存储类负责把内存模型保存到文件。

2 继承 多态与群策略

SocialServiceLJY 为服务抽象接口，QQServiceLJY、WeChatServiceLJY 和

WeiboServiceLJY 提供各自号码模式与默认群规则。群权限使用策略模式：GroupManagerLJY 根据群的 style 取得 GroupPolicyLJY 基类引用，再调用虚函数，由 QQGroupPolicyLJY 或 WeChatGroupPolicyLJY 判断具体权限。图 1 展示这一关键关系。

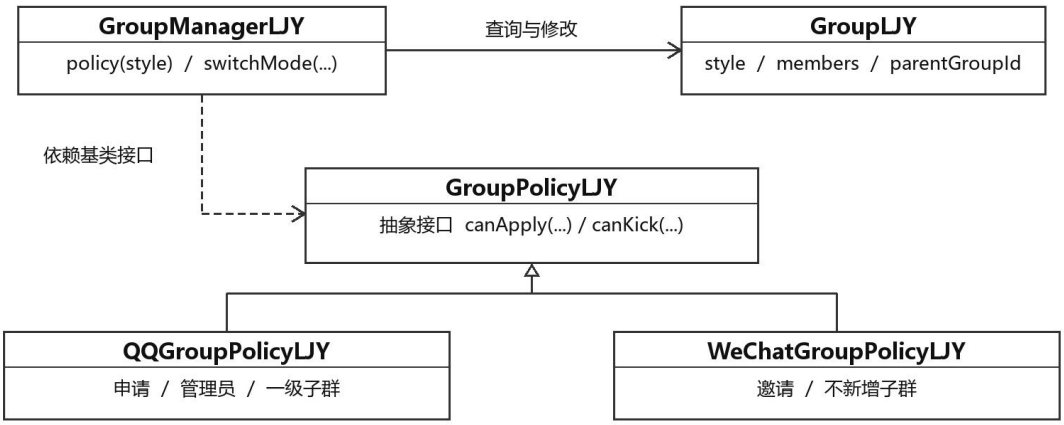


图 1　群策略核心类关系

QQ 模式允许申请入群、管理员审批和一级子群；微信模式采用邀请加入，不提供管理员任免和新建子群。切换模式时修改规则选择，保留成员与已有子群，已有子群仍按自身模式管理。原管理员标记也保留，但在微信模式下不赋予管理特权，切回 QQ 后再按原角色生效。微博默认采用 QQ 群规则。

3 界面与交互组织

图形界面按消息、群管理、好友列表和我的账号分区。群操作先选群，再选成员和动作；修改资料在页面内展开表单，重要操作使用内置确认卡片。底部固定区域显示结果，失败使用浅红提示条，未读使用红色角标。界面负责交互反馈，最终权限仍由业务层判断。

三、详细设计

1 编译环境与核心业务流程

项目采用 C++17、CMake 和 Ninja。图形发布版使用 Qt 6.8.3 与 MinGW 13.1；SQLite 的 C 源码随项目编译为静态库，无需单独安装数据库服务。程序入口创建 PlatformLJY、加载数据并建立会话，再运行控制台菜单或 Qt 事件循环。普通新库初始化演示数据，已有库恢复自身数据；内存模式不写入数据库。

注册先校验资料和两次密码输入，再创建人员及账号，成功后显示登录号码。开通服务时依据号码模式复用 QQ/微博号码或分配微信号码。好友申请只有在对方同意后才生成双向关系；备注保存在本人的关系记录中。群操作通过当前账号定位群内角色，再调用策略校验并修改容器，例如 QQ 模式下，群主移除管理员前必须先取消其管理员身份。

2 容器与持久化实现

PlatformLJY 使用 std::map 按内部编号组织实体与关系，群成员使用 std::vector 保存。编号用于关联对象，私有数据通过查询接口和业务方法访问。std::unique_ptr 管理服务多态对象；线程、Socket 和数据库连接在所属类中统一管理生命周期。

GUI 提交成功后发出 changed 信号，由主窗口调用保存回调；未提交的表单仍是草稿。控制台可通过保存菜单或主菜单正常退出写库，失败会提示并允许重试。SqliteStorageLJY 在事务中保存业务快照，失败回滚，且不清空聊天表。ChatHistoryLJY 和 GroupChatHistoryLJY 分别增量保存私聊、群聊记录；GroupRoomStoreLJY 保存独立群服务日志。业务表、客户端历史与服务端日志职责分开，避免修改备注时覆盖聊天记录。

3 私聊与群聊通信流程

私聊通过 TcpPeerLJY 调用 Winsock，在后台 std::thread 中监听、连接和收发。协议使用四字节大端长度前缀，接收缓冲区保留未完整的数据帧，并能拆出一次接收中的多帧。工作线程把结果放入受互斥量保护的事件队列，界面定时取出后更新气泡并保存历史，避免网络等待阻塞窗口。ACK 只表示对端程序已接收，不代表用户已读。

私聊将正在浏览的好友与实际连接的好友分开保存。切换列表不会断开原连接；只有开始连接新好友才替换旧连接。后台消息仍写入原会话，当前私聊未提供离线投递服务。图 2 为实际消息页，显示会话列表、头像气泡、未读提示与发送区。

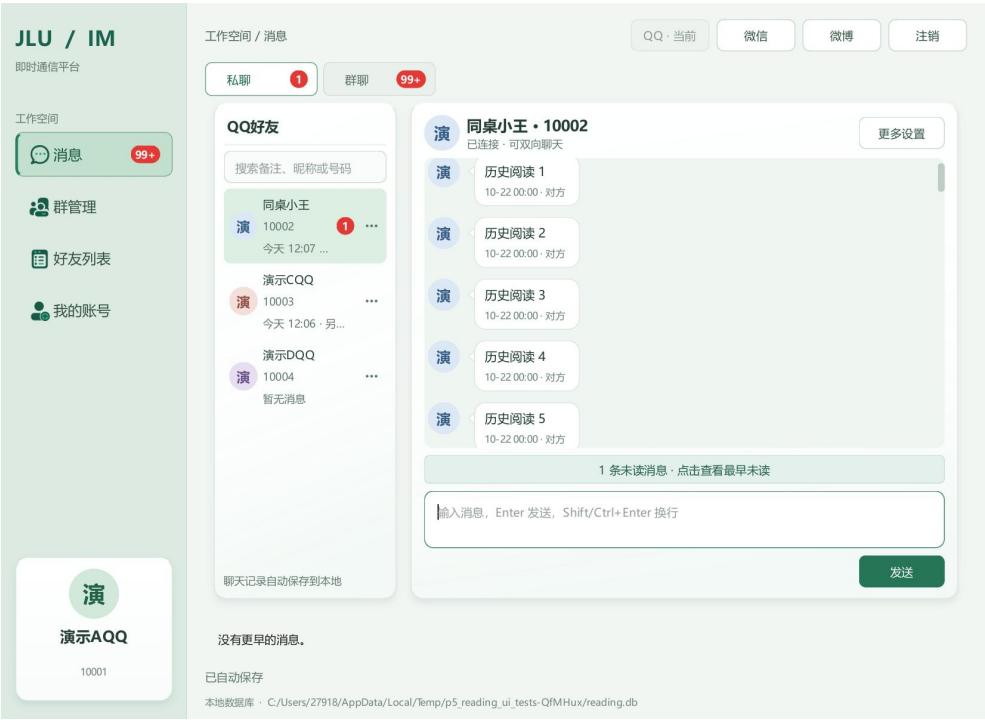


图 2　本机消息界面

群聊由 GroupChatClientLJY 连接独立的 jlu_group_server，后者以 GroupChatHostLJY 管理多个连接，使用 Qt 网络事件循环处理收发。收到消息后先写服

务端日志，再广播给在线成员。客户端加入或恢复连接时按服务端序号分批补齐历史，去重保存到本地；加载更早消息从本地历史分页读取。补齐以群服务已接收并保留日志为前提，服务停止期间不能接收新消息。其他已加入群保持后台接收，因此浏览别的页面不会停止这些群的接收。

4 难点处理与验证

群历史丢失曾表现为离线后回来仍提示“已到最早消息”。原因是本地没有收到离线期间的消息，单纯向前翻页无法获取缺失内容。现由独立群服务持久保存，再通过历史同步补齐。另一类问题是界面浏览对象与网络连接对象混用，通过分别记录两种状态，保持原连接接收并防止向错误对象发送。

既有验收覆盖账号、好友、群权限、持久化、本机聊天与重启恢复。2026 年 10 月 21 日的本机范围调整后，18 项相关测试全部通过，含三服务群离线补齐；10 月 25 日又通过子群选择、保存重载及界面缩放检查。18 项测试的证据为“9A 相关测试_20261021.txt”。本轮核对既有日志和实现，不将本机结果表述为实体双机验收。

四、总结与体会

本项目通过人员、账号、群及关系对象表达业务，通过继承和虚函数处理不同服务与群规则，通过组合把会话、管理器和平台容器连接起来。图形端和终端复用业务核心，使交互形式与数据规则能够分别修改。SQLite 保存与网络收发各自封装后，功能调用路径也更便于追踪。

调试表明，界面上“显示了一个结果”并不等于业务状态和持久化状态一致。例如注册后需要明确显示号码，失败后应保留草稿，离线群消息需要先同步再分页，未读数需要与已读位置一致。这些问题说明，功能设计不仅要考虑一次操作成功，还要说明切换页面、退出和重新启动之后数据应处于什么状态。

现有实现仍有改进空间：图形页面承担较多交互代码，部分辅助类型仍与其他类型共用文件，尚未完全达到每个类独立同名文件的严格约定；当前仅支持 Windows 本机通信，私聊没有离线投递，账号与群业务数据也未统一为远端服务。后续可先整理辅助类型和较大的页面文件，再按实际需要考虑跨机功能。课程交付应优先保证已有功能清楚、稳定且能够解释。